

DAY 8

FUNCTIONS WITH INPUTS

Goals:

- Functions that allows us to give them inputs
- Build a Caesar Cipher
- Difference between Arguments and Parameters

Caesar Cipher

- [What is a Caesar Cipher](#)
- [Caesar Cipher Demo](#)

Functions with Inputs

- Functions are a really handy way of taking a complex set of instructions and packaging them together inside a block of code that has a name giving to it, that we can call with a pair of parenthesis to use at any time
- it's actually rare for you to want to repeat the same set of instructions when you call a function, during such times it's possible to add the name of a variable to a function's parenthesis and giving it a value when calling the function
- **remember to pay attention to the datatype of the variable you pass into a function**
- when using a function with inputs, you can vary it's functionality or value each time you use it
- In programming lingo, the variable created in the parenthesis of function is called an parameter, and the value you give the variable anytime you call it is called an argument

Coding Exercise – Life In Weeks

- [life in weeks article](#)
- Udemy coding exercises do not have console, so you cannot use the input(), so you'd have to call your function with a hard coded input value
- **when using inputs in functions it doesn't have a fixed data type like the input function, so you have to specify as you use**

Positional vs Keyword Arguments

- In this session we'd go a bit further for a function that allows for multiple inputs
- to use multiple inputs in a function all you need to add is a comma: `def greet_with(name, location)`, and then use the parameters in your function, **when calling the function, make sure to be careful of the data type of your arguments and also separate them with the comma when calling**
- You can take as many inputs as you like in your functions
- When the order of your arguments to parameters really matter, this is called a Positional Argument, **so always be careful when using a positional argument**
- For Clarity it's advised to use keyword arguments; `greet_with(location = "Nigeria", name = "Adrian")`, in keyword arguments the order of the arguments to parameters doesn't matter as you've already specified. using keyword arguments definitely makes your code less error prone, although it make each line longer
- no specific argument should always be used, so use your judgement to pick the right one for the right scenario

Coding Exercise – Love Calculator

- You can use the `count()` function count the number of times a character occurs in a string; `string.count("string")`, **you can count both single characters or strings within strings, and it is also case sensitive. `.count()` on a list counts list items, while on a string it counts substrings or characters**

Caesar Cipher Part 1 – Encryption

- [What is a Caesar Cipher](#)
- [Caesar Cipher Demo](#)
- all we'd need is to be able to shift the alphabets by a specific shift number , both forward to encrypt and backward to decrypt
- there's quite some things tho, so it's rather difficult but it's broken down
- the `.index()` function draws out the index of an item on the list
- the output of the `.index()` function is an integer
- you can use a for loop on a string and use each character of the string

Caeser Cipher Part 2 – Decryption

- This is getting our code to decode our text on the other end
- didn't have to do much as i just did the opposite of encrypt, and negative indices is a thing, and since it's counting backwards i don't have to fear it crossing any limit
- try to create new names for your parameters, don't go using a previously created variable can go causing confusion

Caeser Cipher Part 3 -

- when working on the part allowing for special characters; if after you introduce an if statement into the for loop block, you don't indent the rest of code into an else statement, the rest of the code will still consider that particular special character although it's been dealt with in the if block, as there's no condition stopping it's reconsideration and that special character is definitely not on the list, so you're bound to get an error
- we previously considered how to end a while loop from within in previous chapters, and we've done it again. This time we created a variable called; `program_should_run` and equated it to; `True`, and ran the while loop on this variable, and ended the loop using an if condition within the while block that changes the content of the `program_should_run` variable to `False` if user choses not to run it again.
- Remember to use variables to make cleaner and readable code